

实验 2 - 全连接神经网络

数据科学与大数据技术专业

1 实验目标

- 理解全连接神经网络中层、参数、激活值、梯度和缓存 (**cache**) 的表示方式.
- 使用 NumPy 从零实现两层神经网络和 L 层深度神经网络的核心模块.
- 掌握 LINEAR、ReLU、Sigmoid、二元交叉熵、反向传播和梯度下降之间的组合关系.
- 能够将模块化函数组合成完整训练循环, 完成猫/非猫图像二分类任务.
- 比较 Logistic 回归、两层网络和多层网络在图像分类任务上的表现差异, 理解网络深度对表达能力和过拟合风险的影响.
- 训练学生在完成代码填空时记录调试过程、实验现象和 AI 辅助学习过程.

2 实验环境

- 操作系统: Windows / Linux / macOS
- Python 3.7 及以上
- 主要依赖库: `numpy`、`matplotlib`、`h5py`
- 推荐开发环境: Jupyter Notebook 或 VS Code
- 实验文件:

- `W4A1/Building_your_Deep_Neural_Network_Step_by_Step.ipynb`
- `W4A2/Deep Neural Network - Application.ipynb`

如本地环境缺少依赖, 可在终端中执行:

```
pip install numpy matplotlib h5py
```

若使用 Anaconda, 可执行:

```
conda install numpy matplotlib h5py
```

3 背景知识

3.1 全连接神经网络

全连接神经网络 (Fully Connected Neural Network) 由若干线性层和非线性激活函数交替组成. 对第 l 层, 若上一层激活值为 $A^{[l-1]}$, 则该层的线性变换与激活为:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \quad (3.1)$$

$$A^{[l]} = g^{[l]}(Z^{[l]}) \quad (3.2)$$

其中 $W^{[l]}$ 为权重矩阵, $b^{[l]}$ 为偏置向量, $g^{[l]}$ 为激活函数. 输入层记为 $A^{[0]} = X$.

本实验采用的二分类网络结构为:

$$[\text{LINEAR} \rightarrow \text{ReLU}]^{\times(L-1)} \rightarrow \text{LINEAR} \rightarrow \text{Sigmoid} \quad (3.3)$$

隐藏层使用 ReLU 引入非线性, 输出层使用 Sigmoid 将输出映射到 $[0, 1]$ 区间, 表示样本属于正类的概率.

3.2 符号与张量形状

本实验约定一个数据矩阵中每一列表示一个样本. 若共有 m 个样本, 输入特征维度为 n_x , 则:

$$X \in \mathbb{R}^{n_x \times m}, \quad Y \in \mathbb{R}^{1 \times m} \quad (3.4)$$

对于第 l 层, 若该层神经元个数为 $n^{[l]}$, 则:

$$W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}} \quad (3.5)$$

$$b^{[l]} \in \mathbb{R}^{n^{[l]} \times 1} \quad (3.6)$$

$$Z^{[l]}, A^{[l]} \in \mathbb{R}^{n^{[l]} \times m} \quad (3.7)$$

代码实现时要特别检查矩阵乘法方向. 本实验中应使用:

$$Z = \text{np.dot}(W, A_{\text{prev}}) + b \quad (3.8)$$

即由 W 左乘上一层激活值, 而不是让 A_{prev} 左乘 W .

3.3 激活函数

Sigmoid 函数常用于二分类输出层:

$$\sigma(z) = \frac{1}{1 + \exp(-z)} \quad (3.9)$$

当 $\sigma(z) > 0.5$ 时, 通常预测为正类; 否则预测为负类.

ReLU 函数常用于隐藏层:

$$\text{ReLU}(z) = \max(0, z) \quad (3.10)$$

其导数为:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases} \quad (3.11)$$

ReLU 计算简单, 能缓解深层网络中 Sigmoid / Tanh 容易出现的梯度消失问题, 但若学习率过大或初始化不合适, 也可能出现部分神经元长期输出为 0 的现象.

3.4 二元交叉熵损失

对二分类任务, 网络最终输出 $A^{[L]} = \hat{Y}$, 其中 $\hat{y}^{(i)}$ 表示第 i 个样本属于正类的概率. 二元交叉熵损失为:

$$\mathcal{J}(\hat{Y}, Y) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)})] \quad (3.12)$$

实现时应保证 $\hat{Y}$ 与 Y 的形状一致, 通常均为 $(1, m)$. 若预测概率非常接近 0 或 1, 可能出现 $\log(0)$ 的数值问题; 在工程实现中可使用裁剪或更稳定的损失函数形式.

3.5 前向传播与缓存

前向传播不仅要计算输出, 还要保存反向传播需要的中间量. 本实验中每个线性加激活模块保存:

$$\text{linear_cache} = (A^{[l-1]}, W^{[l]}, b^{[l]}) \quad (3.13)$$

$$\text{activation_cache} = Z^{[l]} \quad (3.14)$$

完整的 L 层前向传播会把每一层的 **cache** 按顺序存入列表 **caches**. 反向传播时再从最后一层开始反向读取这些缓存.

3.6 反向传播

反向传播利用链式法则从损失函数对输出的梯度开始, 逐层计算各参数梯度. 对线性部分:

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} (A^{[l-1]})^T \quad (3.15)$$

$$db^{[l]} = \frac{1}{m} \sum_{i=1}^m dZ^{[l](i)} \quad (3.16)$$

$$dA^{[l-1]} = (W^{[l]})^T dZ^{[l]} \quad (3.17)$$

对激活函数部分:

$$dZ^{[l]} = dA^{[l]} \odot g'^{[l]}(Z^{[l]}) \quad (3.18)$$

其中 $\odot$ 表示逐元素乘法. 对输出层 Sigmoid + 交叉熵, 首先计算:

$$dA^{[L]} = -\frac{Y}{A^{[L]}} + \frac{1 - Y}{1 - A^{[L]}} \quad (3.19)$$

然后再调用 Sigmoid 的反向传播函数得到 $dZ^{[L]}$.

3.7 参数初始化与梯度下降

训练前必须打破神经元之间的对称性. 若所有权重初始化为 0, 同一层神经元会得到相同梯度, 无法学习不同特征. 本实验第一部分的评分函数要求:

$$W^{[l]} \sim \mathcal{N}(0, 1) \times 0.01, \quad b^{[l]} = \mathbf{0} \quad (3.20)$$

在应用部分, 深层网络可使用按输入维度缩放的随机初始化:

$$W^{[l]} \sim \frac{\mathcal{N}(0, 1)}{\sqrt{n^{[l-1]}}} \quad (3.21)$$

参数更新使用普通梯度下降:

$$W^{[l]} \leftarrow W^{[l]} - \alpha dW^{[l]} \quad (3.22)$$

$$b^{[l]} \leftarrow b^{[l]} - \alpha db^{[l]} \quad (3.23)$$

其中 α 为学习率. 学习率过小会导致收敛慢, 过大则可能导致损失震荡甚至发散.

3.8 图像分类数据预处理

猫/非猫数据集中每张图像大小为 $64 \times 64 \times 3$. 神经网络输入需要将其展平为列向量:

$$64 \times 64 \times 3 = 12288 \quad (3.24)$$

因此训练集输入形状应整理为:

$$X_{\text{train}} \in \mathbb{R}^{12288 \times m_{\text{train}}} \quad (3.25)$$

同时, 像素值从 $[0, 255]$ 归一化到 $[0, 1]$:

$$X \leftarrow \frac{X}{255} \quad (3.26)$$

归一化可以让各特征处在相近尺度上, 通常能加快梯度下降收敛.

4 实验步骤

实验 2.1: 深层神经网络模块化实现 (W4A1)

实验目标:

- 在 W4A1 Notebook 中逐个实现深度神经网络的基础函数.
- 完成后使用 `public_tests.py` 中的单元测试检查结果.

2.1.1 参数初始化

练习 1: initialize_parameters

实现两层神经网络的初始化函数. 网络结构为输入层 $\rightarrow$ 隐藏层 $\rightarrow$ 输出层, 参数形状如下:

- $W^{[1]} \in \mathbb{R}^{n_h \times n_x}, b^{[1]} \in \mathbb{R}^{n_h \times 1}$
- $W^{[2]} \in \mathbb{R}^{n_y \times n_h}, b^{[2]} \in \mathbb{R}^{n_y \times 1}$

实现要点:

- 使用 `np.random.seed(1)` 保证测试结果可复现.
- 权重使用 `np.random.randn(...)*0.01`.
- 偏置使用 `np.zeros(...)`.

练习 2: initialize_parameters_deep

将初始化推广到任意层数. 输入 `layer_dims` 是一个列表, 如 `[5,4,3]` 表示输入层 5 维、隐藏层 4 个神经元、输出层 3 维. 对第 l 层:

$$W^{[l]} \in \mathbb{R}^{\text{layer_dims}[l] \times \text{layer_dims}[l-1]}, \quad b^{[l]} \in \mathbb{R}^{\text{layer_dims}[l] \times 1} \quad (4.1)$$

参数字典键名统一为 `W1`、`b1`、...、`WL`、`bL`.

2.1.2 前向传播模块

练习 3: linear_forward

实现单层线性变换:

$$Z = WA + b \quad (4.2)$$

函数返回 `Z` 以及 `cache=(A,W,b)`.

练习 4: linear_activation_forward

将线性变换和激活函数组合为一个模块:

$$A = g(WA_{\text{prev}} + b) \quad (4.3)$$

本实验只需要支持两种字符串参数:

- `"relu"`: 隐藏层激活函数.
- `"sigmoid"`: 输出层激活函数.

函数返回激活值 `A` 和 `cache=(linear_cache, activation_cache)`.

练习 5: L_model_forward

实现完整的 L 层前向传播:

$$[\text{LINEAR} \rightarrow \text{ReLU}]^{\times(L-1)} \rightarrow \text{LINEAR} \rightarrow \text{Sigmoid} \quad (4.4)$$

实现时先循环处理前 $L - 1$ 层 ReLU, 最后单独处理 Sigmoid 输出层. 函数应返回最终概率 `AL` 和包含所有层缓存的 `caches`.

2.1.3 损失函数

练习 6: compute_cost

实现二元交叉熵损失:

$$\mathcal{J} = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log a^{[L](i)} + (1 - y^{(i)}) \log(1 - a^{[L](i)})] \quad (4.5)$$

注意返回值应为标量, 可使用 `np.squeeze(cost)` 去掉多余维度.

2.1.4 反向传播模块

练习 7: linear_backward

已知当前层 dZ , 实现线性部分反向传播:

$$dW = \frac{1}{m} dZ A_{\text{prev}}^T \quad (4.6)$$

$$db = \frac{1}{m} \sum dZ \quad (4.7)$$

$$dA_{\text{prev}} = W^T dZ \quad (4.8)$$

计算 db 时应使用 `axis=1, keepdims=True`, 以保持列向量形状.

练习 8: linear_activation_backward

根据激活函数类型选择对应的反向函数:

- "relu": 调用 `relu_backward`.
- "sigmoid": 调用 `sigmoid_backward`.

得到 dZ 后, 再调用 `linear_backward`. 该函数返回 dA_{prev} 、 dW 、 db .

练习 9: L_model_backward

实现完整的 L 层反向传播. 实现顺序为:

1. 根据二元交叉熵计算 $dA^{[L]}$.
2. 对最后一层调用 Sigmoid 反向传播.
3. 从后向前遍历其余层, 对每一层调用 ReLU 反向传播.
4. 将所有梯度存入 `grads` 字典, 键名如 `dA0`、`dW1`、`db1`.

2.1.5 参数更新

练习 10: update_parameters

使用梯度下降更新每一层参数:

$$W^{[l]} \leftarrow W^{[l]} - \alpha dW^{[l]} \quad (4.9)$$

$$b^{[l]} \leftarrow b^{[l]} - \alpha db^{[l]} \quad (4.10)$$

Notebook 中建议对输入参数字典做深拷贝, 避免测试时原始对象被意外修改.

实验 2.2: 猫/非猫图像分类应用 (W4A2)

实验目标: 使用实验 2.1 中实现的基础函数, 构建完整的训练循环, 在真实图像数据集上比较两层网络和 L 层深度网络.

2.2.1 数据加载与预处理

使用 `load_data()` 加载 H5 数据集. 数据包含训练图像、训练标签、测试图像、测试标签以及类别名称. 每张图像为 $64 \times 64 \times 3$ 的 RGB 图像.

需要完成的预处理:

- 查看训练集和测试集规模, 确认标签形状.
- 将图像从 $(m, 64, 64, 3)$ 展平并转置为 $(12288, m)$.
- 将像素值除以 255, 归一化到 $[0, 1]$.
- 随机查看若干图像, 理解数据集中猫与非猫样本的差异.

2.2.2 两层神经网络

练习 1: `two_layer_model`

构建两层神经网络:

$$\text{LINEAR} \rightarrow \text{ReLU} \rightarrow \text{LINEAR} \rightarrow \text{Sigmoid} \quad (4.11)$$

推荐维度设置:

$$n_x = 12288, \quad n_h = 7, \quad n_y = 1 \quad (4.12)$$

训练循环包含:

1. 调用 `initialize_parameters` 初始化参数.
2. 调用 `linear_activation_forward` 完成两层前向传播.
3. 调用 `compute_cost` 计算损失.
4. 手动计算输出层初始梯度 $dA^{[2]}$.
5. 调用 `linear_activation_backward` 依次完成输出层和隐藏层反向传播.
6. 调用 `update_parameters` 更新参数.
7. 每 100 次迭代记录一次损失, 用于绘制损失曲线.

参考设置为学习率 0.0075, 训练 2500 次迭代. Notebook 中测试集准确率通常约为 72%.

2.2.3 L 层深度神经网络

练习 2: L_layer_model

构建更深的网络:

$$[\text{LINEAR} \rightarrow \text{ReLU}]^{\times(L-1)} \rightarrow \text{LINEAR} \rightarrow \text{Sigmoid} \quad (4.13)$$

Notebook 中推荐的层维度为:

$$[12288, 20, 7, 5, 1] \quad (4.14)$$

训练循环比两层模型更简洁, 因为完整前向传播和反向传播已经封装为:

- `L_model_forward(X, parameters)`
- `L_model_backward(AL, Y, caches)`

参考设置同样为学习率 0.0075, 训练 2500 次迭代. Notebook 中测试集准确率通常约为 80%, 高于两层网络.

2.2.4 结果分析

完成训练后, 至少记录以下结果:

- 两层网络训练集准确率与测试集准确率.
- L 层网络训练集准确率与测试集准确率.
- 两个模型的损失曲线是否稳定下降.
- 错误分类图片的典型情况, 如背景干扰、光线过暗、猫的姿态特殊、猫在图中占比过小等.

分析时要注意: 若训练集准确率明显高于测试集准确率, 说明模型可能已经对训练样本产生过拟合. 更深的网络表达能力更强, 但并不保证测试集准确率无限提升.

2.2.5 自定义图片测试 (选做)

可将自己的图片放入 `images/` 文件夹, 修改 Notebook 中的 `my_image` 和 `my_label_y`, 使用训练好的模型预测图片中是否有猫. 图片应尽量处理为 RGB 格式, 并按照 Notebook 中的方式缩放、展平、归一化.

5 核心函数对照总结

函数	所在实验	作用
<code>initialize_parameters</code>	W4A1 / W4A2	初始化两层网络参数
<code>initialize_parameters_deep</code>	W4A1 / W4A2	初始化 L 层网络参数
<code>linear_forward</code>	W4A1	计算 $Z = WA + b$
<code>linear_activation_forward</code>	W4A1	组合线性层与激活函数
<code>L_model_forward</code>	W4A1	完整 L 层前向传播
<code>compute_cost</code>	W4A1	计算二元交叉熵损失
<code>linear_backward</code>	W4A1	计算线性层梯度
<code>linear_activation_backward</code>	W4A1	组合激活反向与线性反向
<code>L_model_backward</code>	W4A1	完整 L 层反向传播
<code>update_parameters</code>	W4A1	梯度下降更新参数
<code>two_layer_model</code>	W4A2	训练两层猫/非猫分类器
<code>L_layer_model</code>	W4A2	训练 L 层猫/非猫分类器

6 实验作业

1. 完成 W4A1 Notebook 中所有代码填空, 包括 10 个核心函数, 并确保所有公开测试通过.
2. 完成 W4A2 Notebook 中 `two_layer_model` 与 `L_layer_model` 的实现, 能够完成训练、预测和损失曲线绘制.
3. 在实验记录中比较两层网络和 L 层网络的训练集准确率、测试集准确率与损失下降曲线.
4. 分析至少 3 张错误分类图片, 说明可能的错误原因.
5. 回答以下思考题:
 - (a) 为什么隐藏层使用 ReLU, 输出层使用 Sigmoid?
 - (b) 为什么权重不能全部初始化为 0?
 - (c) 为什么图像输入要展平并除以 255?
 - (d) 如果训练集准确率接近 100%, 但测试集准确率明显较低, 说明了什么?
 - (e) 继续增加网络层数是否一定能提高测试准确率? 为什么?
6. **选做:** 调整隐藏层维度、学习率或训练迭代次数, 观察对训练稳定性和测试准确率的影响.
7. **选做:** 尝试使用 Tanh 替换部分隐藏层 ReLU, 比较损失曲线和准确率变化.
8. **选做:** 使用自己的图片测试模型, 并记录预测结果与分析.

7 提交要求

请在提交前检查材料是否齐全. 本次实验提交内容包括 Notebook 源文件和实验过程记录. 本课程作业上传只接受文本文件, 不接受 PDF、Word 文档、图片、截图、压缩包等二进制文件. 采用这一规则, 一方面是为了引入 AI 辅助评阅, 另一方面也是希望大家养成用清晰文本表达实验过程和结论的习惯.

如果实验报告确实需要配图, 请优先思考是否可以用文字、公式或表格表达清楚; 若必须插图, 请将图片放入 Notebook 中, 形成一份完整的 `.ipynb` 实验报告. 若使用 $\text{\LaTeX}$ 或 Word 撰写报告, 请使用 `pandoc` 等工具转换为 `.md` 或 `.txt` 等文本格式后再提交, 不要提交 `.pdf` 或 `.docx`.

1. Jupyter Notebook 文件 (.ipynb)

- (a) 必交内容包括 W4A1 和 W4A2 对应的 Notebook 文件.
- (b) Notebook 中应包含完整代码、运行结果、损失曲线、准确率输出和必要的分析文字.
- (c) 只需提交 Notebook 源文件, 不需要额外导出为 PDF、HTML、Python 脚本、Word 文档、截图或图片文件.
- (d) 若完成选做内容, 请在 Notebook 中保留相关代码、运行结果和简要说明.

2. 实验过程与 AI 互动记录文本文件 (.md / .txt)

- (a) 必须额外提交一份文本文件, 作为实验过程与 AI 互动记录, 推荐使用 `.md` 或 `.txt`.
- (b) 记录内容至少包括: 查阅了哪些资料、向 AI 提出了哪些问题、如何继续思考、调试、修改代码或完善实验分析.
- (c) 不要把 AI 的回复原文放入记录中, 也不要复制粘贴大段代码; 记录应只包含你自己写的内容, 包括你的问题、判断、尝试、修改思路和必要的简短总结.
- (d) 记录应尽量按时间顺序整理, 不要只保留最终答案.
- (e) 建议文件命名为 AI 问答记录 `.md` 或 实验过程记录 `.md`.

8 关于作业的重要说明

本实验的核心目标不是得到一份表面上完整的代码, 而是理解深度神经网络训练过程中的每一个基础模块. 因此, 代码填空部分应由学生自己完成, 并通过调试和单元测试逐步确认正确性.

可以使用 AI 工具辅助理解概念、解释报错、查询 API 用法或检查矩阵维度, 但不得直接让 AI 代写全部必做代码后不加理解地提交. 若使用 AI 工具, 必须在文本记录中如实写明自己的提问、自己的判断和后续修改过程.

若最终提交材料只有完整代码, 缺少实验过程、运行结果、调试记录和必要分析, 将难以证明实验由本人独立完成, 作业评分会受到明显影响.

AI 互动记录应只保留学生自己写的内容. 不要粘贴 AI 的完整回复, 不要粘贴复制来的大段代码, 也不要聊天记录原样导出后提交. 记录的重点是说明你如何提出问题.

选做内容可以更自由地使用 AI 工具, 不需记录使用过程.